04. dstack 키 유도 및 관리 시스템 상세 분석

목차

1. 키 관리 시스템 개요
2. KMS 키 계층 구조
3. 애플리케이션 키 유도
4. Guest Agent 키 처리
5. 암호화 키 타입별 분석
6. 키 서명 체인
7. 환경 변수 암호화
8. TLS 인증서 키 관리

---

1. 키 관리 시스템 개요

1.1 시스템 아키텍처

```
┌─────────────────────────────────────────────────────────────┐
│ KMS (Key Management Service) │
├─────────────────────────────────────────────────────────────┤
│ Root CA Key ──┬── App CA Key (per app) │
│ ├── App Disk Encryption Key │
```

```
│ ├── Environment Encryption Key │
│ └── K256 App Key │
├──────────────────────────────────────────────────────────────┤
│ K256 Root Key ─── K256 App Key (with signature chain) │
└──────────────────────────────────────────────────────────────┘
│
▼
┌──────────────────────────────────────────────────────────────┐
│ Guest Agent │
├──────────────────────────────────────────────────────────────┤
│ App Keys ──┬── TLS Certificate Keys (derived) │
│ ├── Application-specific Keys (path-based) │
│ └── Signature Verification │
└──────────────────────────────────────────────────────────────┘
```

1.2 키 유도 원칙

dstack의 키 관리는 **계층적 결정론적 키 유도(Hierarchical Deterministic Key Derivation)**를 기반으로 합니다:

1. **Root Key**: KMS에서 관리하는 마스터 키
2. **Context-based Derivation**: 애플리케이션 ID, 인스턴스 ID, 용도별 컨텍스트 사용
3. **HKDF-SHA256**: 표준 키 유도 함수 사용
4. **Signature Chain**: 키의 출처와 무결성 보장

---

2. KMS 키 계층 구조

2.1 KMS 상태 구조

```
// kms/src/main_service.rs (추정)
pub struct KmsState {
pub root_ca: CaCert, // Root CA 키 쌍
pub k256_key: SigningKey, // K256 루트 키
pub temp_ca_key: String, // 임시 CA 키
pub config: KmsConfig, // KMS 설정
}

pub struct CaCert {
pub key: KeyPair, // CA 개인키
pub pem_cert: String, // CA 인증서 (PEM)
}
```

2.2 애플리케이션 키 요청 처리

```
// kms/src/main_service.rs
impl KmsRpc for RpcHandler {
async fn get_app_key(self, request: GetAppKeyRequest) -> Result<AppKeyResponse
// 1. API 버전 확인
if request.api_version > 1 {
```

```
bail!("Unsupported API version: {}", request.api_version);

}

// 2. 애플리케이션 부팅 권한 확인

let BootConfig {

boot_info,

gateway_app_id,

os_image_hash,

} = self

.ensure_app_boot_allowed(&request.vm_config)

.await

.context("App not allowed")?;

let  app_id = boot_info.app_id;

let  instance_id = boot_info.instance_id;



// 3. 디스크 암호화 키 유도

let  context_data = vec![&app_id[..], &instance_id[..], b"app-disk-crypt-key"];

let  app_disk_key = kdf::derive_dh_secret(&self.state.root_ca.key, &context_data

.context("Failed to derive app disk key")?;

// 4. 환경 변수 암호화 키 유도

let  env_crypt_key = {

let  secret =

kdf::derive_dh_secret(&self.state.root_ca.key, &[&app_id[..], b"env-encrypt-key"

.context("Failed to derive env encrypt key")?;
```

```
let  secret = x25519_dalek::StaticSecret::from(secret);
secret.to_bytes()
};

// 5. K256 애플리케이션 키 유도
let (k256_key, k256_signature) = {
let (k256_app_key, signature) = derive_k256_key(&self.state.k256_key, &app_id)
.context("Failed to derive app ecdsa key")?;
(k256_app_key.to_bytes().to_vec(), signature)
};

// 6. 응답 구성
Ok(AppKeyResponse {
ca_cert: self.state.root_ca.pem_cert.clone(),
disk_crypt_key: app_disk_key.to_vec(),
env_crypt_key: env_crypt_key.to_vec(),
k256_key,
k256_signature,
tproxy_app_id: gateway_app_id.clone(),
gateway_app_id,
os_image_hash,
})
}
```

```
}
```

---

3. 애플리케이션 키 유도

3.1 K256 키 유도 과정

```
// kms/src/crypto.rs

pub(crate) fn derive_k256_key(

parent_key: &SigningKey,

app_id: &[u8],

) -> Result<(SigningKey, Vec<u8>)> {

// 1. 컨텍스트 데이터 구성

let context_data = [app_id, b"app-key"];

// 2. HKDF를 사용한 키 유도

let derived_key_bytes: [u8; 32] =

kdf::derive_ecdsa_key(&parent_key.to_bytes(), &context_data, 32)?

.try_into()

.ok()

.context("Invalid derived key len")?;

// 3. SigningKey 생성

let derived_signing_key = SigningKey::from_bytes(&derived_key_bytes.into())?;

let pubkey = derived_signing_key.verifying_key();
```

```
// 4. 서명 체인 생성
let signature = sign_message(
parent_key,
b"dstack-kms-issued",
app_id,
&pubkey.to_sec1_bytes(),
)?;
Ok((derived_signing_key, signature))
}
```

3.2 HKDF 키 유도 함수

```
// ra-tls/src/kdf.rs
/// Derives a key using HKDF-SHA256.
pub fn derive_ecdsa_key(
input_key_material: &[u8],
context_data: &[&[u8]],
key_size: usize,
) -> Result<Vec<u8>, Unspecified> {
// 1. Salt 설정 ("RATLS" 고정값 사용)
let salt = Salt::new(HKDF_SHA256, b"RATLS");
// 2. Extract 단계: PRK (Pseudo-Random Key) 생성
```

```
let pseudo_rand_key: Prk = salt.extract(input_key_material);

// 3. Expand 단계: 최종 키 생성

let output_key_material: Okm<AnySizeKey> =

pseudo_rand_key.expand(context_data, AnySizeKey(key_size))?;

// 4. 결과 버퍼에 키 데이터 채우기

let mut result = vec![0u8; key_size];

output_key_material.fill(&mut result)?;

Ok(result)

}
```

3.3 DH Secret 유도

```
// ra-tls/src/kdf.rs

/// Derives a X25519 secret from a given key pair.

pub fn derive_dh_secret(from: &KeyPair, context_data: &[&[u8]]) -> Result<[u8;

// 1. ECDSA 키 쌍 유도

let key_pair = derive_ecdsa_key_pair(from, context_data)?;

// 2. SHA256 해시를 통한 DH Secret 생성

let derived_secret = sha256(key_pair.serialized_der());

Ok(derived_secret)

}


fn sha256(data: &[u8]) -> [u8; 32] {
```

```rust
use sha2::{Digest, Sha256};

let mut hasher = Sha256::new();

hasher.update(data);

hasher.finalize().into()

}
```

---

4. Guest Agent 키 처리

4.1 AppKeys 구조

```rust
// dstack_types/src/lib.rs (추정)

#[derive(Serialize, Deserialize)]

pub struct AppKeys {

pub k256_key: Vec<u8>, // K256 개인키

pub k256_signature: Vec<u8>, // K256 서명 체인

pub ca_cert: String, // CA 인증서

pub disk_crypt_key: Vec<u8>, // 디스크 암호화 키

pub env_crypt_key: Vec<u8>, // 환경 변수 암호화 키

}
```

4.2 키 요청 처리

```rust
// guest-agent/src/rpc_service.rs
```

```
impl DstackGuestRpc for InternalRpcHandler {

async  fn  get_key(self, request: GetKeyArgs) -> Result<GetKeyResponse> {

let  k256_app_key = &self.state.inner.keys.k256_key;

// 1. 경로 기반 키 유도

let  derived_k256_key = derive_ecdsa_key(

k256_app_key,

&[request.path.as_bytes()],

32

).context("Failed to derive k256 key")?;

// 2. SigningKey 생성

let  derived_k256_key = SigningKey::from_slice(&derived_k256_key)

.context("Failed to parse k256 key")?;

let  derived_k256_pubkey = derived_k256_key.verifying_key();

// 3. 서명 체인 생성

let  msg_to_sign = format!(

"{}:{}",

request.purpose,

hex::encode(derived_k256_pubkey.to_sec1_bytes())

);

let  app_signing_key = SigningKey::from_slice(k256_app_key)

.context("Failed to parse app k256 key")?;

let  digest = Keccak256::new_with_prefix(msg_to_sign);

let (signature, recid) = app_signing_key.sign_digest_recoverable(digest)?;
```

```
let mut signature = signature.to_vec();
signature.push(recid.to_byte());


Ok(GetKeyResponse {
key: derived_k256_key.to_bytes().to_vec(),
signature_chain: vec![signature, self.state.inner.keys.k256_signature.clone()],
})
}
}
```

---

5. 암호화 키 타입별 분석

5.1 디스크 암호화 키

```
// KMS에서 생성
let context_data = vec![&app_id[..], &instance_id[..], b"app-disk-crypt-key"];
let app_disk_key = kdf::derive_dh_secret(&self.state.root_ca.key, &context_data
.context("Failed to derive app disk key")?;
```

특징:

- **용도**: 애플리케이션 데이터 디스크 암호화
- **컨텍스트**: App ID + Instance ID + “app-disk-crypt-key”

- **알고리즘**: X25519 DH Secret (32바이트)
- **유일성**: 인스턴스별로 고유

5.2 환경 변수 암호화 키

```
// KMS에서 생성
let secret = kdf::derive_dh_secret(&self.state.root_ca.key, &[&app_id[..], b"en
.context("Failed to derive env encrypt key")?;
let secret = x25519_dalek::StaticSecret::from(secret);
let env_crypt_key = secret.to_bytes();
```

특징:

- **용도**: 환경 변수 암호화/복호화
- **컨텍스트**: App ID + “env-encrypt-key”
- **알고리즘**: X25519 Static Secret (32바이트)
- **유일성**: 애플리케이션별로 고유

5.3 K256 애플리케이션 키

```
// KMS에서 생성
let (k256_app_key, signature) = derive_k256_key(&self.state.k256_key, &app_id)
.context("Failed to derive app ecdsa key")?;
```

특징:

- **용도**: 디지털 서명, 키 유도의 부모 키

- **컨텍스트**: App ID + “app-key”
- **알고리즘**: secp256k1 ECDSA (32바이트)
- **서명 체인**: KMS 루트 키로 서명됨

---

6. 키 서명 체인

6.1 서명 생성 과정

```
// kms/src/crypto.rs
pub(crate) fn sign_message(
key: &SigningKey,
prefix: &[u8],
appid: &[u8],
message: &[u8],
) -> Result<Vec<u8>> {
// 1. 메시지 구성: prefix:appid:message
let digest = Keccak256::new_with_prefix([prefix, b":", appid, message].concat()
// 2. 복구 가능한 서명 생성
let (signature, recid) = key.sign_digest_recoverable(digest)?;
// 3. 서명 + 복구 ID 결합
let mut signature_bytes = signature.to_vec();
signature_bytes.push(recid.to_byte());
Ok(signature_bytes)
```

```
}
```

6.2 서명 체인 구조

```
KMS Root K256 Key
|
├─ sign("dstack-kms-issued", app_id, app_pubkey)
|
▼
App K256 Key
|
├─ sign("purpose", derived_pubkey)
|
▼
Derived Key (for specific purpose)
```

6.3 서명 검증 과정

```
// 서명 체인 검증 (의사 코드)
fn  verify_signature_chain(
derived_key: &[u8],
signature_chain: &[Vec<u8>],
app_id: &[u8],
```

```
purpose: &str,
) -> Result<bool> {
// 1. 유도된 키의 공개키 계산
let derived_pubkey = derive_public_key(derived_key)?;
// 2. 첫 번째 서명 검증 (App Key → Derived Key)
let msg1 = format!("{}:{}", purpose, hex::encode(&derived_pubkey));
verify_keccak256_signature(&signature_chain[0], &msg1, app_pubkey)?;
// 3. 두 번째 서명 검증 (KMS Root → App Key)
let msg2 = format!("dstack-kms-issued:{}", hex::encode(app_pubkey));
verify_keccak256_signature(&signature_chain[1], &msg2, kms_root_pubkey)?;
Ok(true)
}
```

---

7. 환경 변수 암호화

7.1 공개키 요청

```
// kms/src/main_service.rs
async fn get_app_env_encrypt_pub_key(self, request: AppId) -> Result<PublicKey
// 1. 환경 변수 암호화용 비밀키 유도
let secret = kdf::derive_dh_secret(
&self.state.root_ca.key,
&[&request.app_id[..], "env-encrypt-key".as_bytes()],
```

```
)
.context("Failed to derive env encrypt key")?;
// 2. X25519 키 쌍 생성
let secret = x25519_dalek::StaticSecret::from(secret);
let pubkey = x25519_dalek::PublicKey::from(&secret);


// 3. 공개키 서명
let public_key = pubkey.to_bytes().to_vec();
let signature = sign_message(
&self.state.k256_key,
b"dstack-env-encrypt-pubkey",
&request.app_id,
&public_key,
)
.context("Failed to sign the public key")?;


Ok(PublicKeyResponse {
public_key,
signature,
})
}
```

7.2 암호화/복호화 과정

```rust
// 클라이언트 측 암호화 (의사 코드)
fn encrypt_env_var(plaintext: &str, app_pubkey: &[u8; 32]) -> Result<Vec<u8>> {
// 1. 임시 키 쌍 생성
let ephemeral_secret = x25519_dalek::StaticSecret::new(&mut OsRng);
let ephemeral_public = x25519_dalek::PublicKey::from(&ephemeral_secret);
// 2. 공유 비밀 계산
let app_public = x25519_dalek::PublicKey::from(*app_pubkey);
let shared_secret = ephemeral_secret.diffie_hellman(&app_public);
// 3. 암호화 키 유도
let encryption_key = hkdf_expand(shared_secret.as_bytes(), b"env-encrypt", 32)?
// 4. AES-GCM 암호화
let ciphertext = aes_gcm_encrypt(plaintext.as_bytes(), &encryption_key)?;
// 5. 임시 공개키 + 암호문 결합
let mut result = ephemeral_public.to_bytes().to_vec();
result.extend_from_slice(&ciphertext);
Ok(result)
}

// Guest Agent 복호화 (의사 코드)
fn decrypt_env_var(ciphertext: &[u8], app_secret: &[u8; 32]) -> Result<String>
// 1. 임시 공개키 추출
```

```
let ephemeral_public = x25519_dalek::PublicKey::from(ciphertext[..32].try_into(
let encrypted_data = &ciphertext[32..];
// 2. 공유 비밀 계산
let app_secret = x25519_dalek::StaticSecret::from(*app_secret);
let shared_secret = app_secret.diffie_hellman(&ephemeral_public);
// 3. 복호화 키 유도
let decryption_key = hkdf_expand(shared_secret.as_bytes(), b"env-encrypt", 32)?
// 4. AES-GCM 복호화
let plaintext = aes_gcm_decrypt(encrypted_data, &decryption_key)?;
Ok(String::from_utf8(plaintext)?)
}
```

---

8. TLS 인증서 키 관리

8.1 TLS 키 생성

```
// guest-agent/src/rpc_service.rs
impl DstackGuestRpc for InternalRpcHandler {
async fn get_tls_key(self, request: GetTlsKeyArgs) -> anyhow::Result<GetTlsKey
// 1. 보안 시드 생성
let mut seed = [0u8; 32];
SystemRandom::new()
.fill(&mut seed)
```

```
.context("Failed to generate secure seed")?;

// 2. ECDSA 키 쌍 유도

let  derived_key =

derive_ecdsa_key_pair_from_bytes(&seed, &[]).context("Failed to derive key")?;

// 3. 인증서 설정

let  config = CertConfig {

org_name: None,

subject: request.subject,

subject_alt_names: request.alt_names,

usage_server_auth: request.usage_server_auth,

usage_client_auth: request.usage_client_auth,

ext_quote: request.usage_ra_tls,

};

// 4. 인증서 요청

let  certificate_chain = self

.state

.inner

.cert_client

.request_cert(&derived_key, config, self.state.config().simulator.enabled)

.await

.context("Failed to sign the CSR")?;

Ok(GetTlsKeyResponse {

key: derived_key.serialize_pem(),
```

```
certificate_chain,
})
}
}
```

8.2 RA-TLS 인증서

```
// cert-client/src/lib.rs (추정)
impl CertRequestClient {
pub  async  fn  request_cert(
&self,
key: &KeyPair,
config: CertConfig,
simulator: bool,
) -> Result<Vec<String>> {
// 1. CSR 생성
let  csr = generate_csr(key, &config)?;
// 2. Quote 생성 (RA-TLS인 경우)
let (quote, event_log) = if  config.ext_quote && !simulator {
let  pubkey = key.public_key_der();
let  report_data = QuoteContentType::RaTlsCert.to_report_data(&pubkey);
let (_, quote) = get_quote(&report_data, None).context("Failed to get quote")?;
```

```
let event_log = read_event_logs().context("Failed to decode event log")?;

let event_log = serde_json::to_string(&event_log)

.context("Failed to serialize event log")?;

(Some(quote), Some(event_log))

} else {

(None, None)

};

// 3. 인증서 서명 요청

let cert = self.sign_certificate(csr, quote, event_log).await?;

Ok(vec![cert])

}

}
```

요약

dstack의 키 유도 및 관리 시스템은 다음과 같은 특징을 가집니다:

핵심 특징:

1. **계층적 키 구조**: Root Key → App Key → Derived Key
2. **결정론적 유도**: 동일한 입력에 대해 항상 동일한 키 생성
3. **컨텍스트 기반**: 용도별, 애플리케이션별 고유 컨텍스트 사용
4. **서명 체인**: 키의 출처와 무결성 보장
5. **다양한 암호화 알고리즘**: ECDSA, X25519, AES-GCM 지원

키 타입별 용도:

- **디스크 암호화 키**: 애플리케이션 데이터 보호
- **환경 변수 암호화 키**: 민감한 설정 정보 보호
- **K256 애플리케이션 키**: 디지털 서명 및 추가 키 유도
- **TLS 인증서 키**: 네트워크 통신 보안

보안 특성:

- **TEE 격리**: 키 유도 과정이 TEE 내부에서 수행
- **하드웨어 증명**: TDX Quote를 통한 키 생성 환경 증명
- **Forward Secrecy**: 임시 키를 사용한 환경 변수 암호화
- **Non-repudiation**: 서명 체인을 통한 키 출처 추적

이 시스템을 통해 dstack은 TEE 환경의 보안성을 활용하면서도 유연하고 확장 가능한 키 관리를 제공합니다.